Agile4MLS—Leveraging Agile Practices for Developing Machine Learning-Enabled Systems

An Industrial Experience

Karthik Vaidhyanathan, IIIT Hyderabad and University of L'Aquila

Anish Chandran, Founding Minds

Henry Muccini, University of L'Aquila

Regi Roy, Founding Minds

// *Increased adoption of machine learning (ML) in business has led to a surge in ML-enabled systems in the market giving rise to new challenges. This article provides insights into Agile4MLS, highlighting the challenges we faced and the process we followed.* //

A SIGNIFICANT INCREASE in the adoption of machine learning (ML) in business has led to the emergence of ML-enabled systems.[1] Although this is the case, ML-enabled systems introduce additional challenges, which has resulted in many systems being shelved as opposed to being deployed in production.[2,3] Lately, practitioners and the academic research community have been working toward identifying and solving some of those challenges related to ML development,[4,5] deployment,[6,7] and testing.[8] However, not much work has been done on the process aspects of developing ML-enabled systems.[9,10] In 2020, we at Founding Minds (FMS) (https://www.foundingminds.com) faced such a challenge. FMS is a small, medium-sized enterprise that provides different types of software (SW) development services for financial, health care, retail, hospitality, and logistics domains.

Since 2018, FMS has been developing a computer vision-based solution to track employee attendance in an organization. Based on this, a few large organizations in India (those with more than 10,000 employees) approached FMS to develop a system that guarantees a secure virtual

Digital Object Identifier 10.1109/MS.2022.3195432
Date of current version: 24 October 2022

workspace. However, the timeline was limited due to business competition and demand. As the organization was primarily an agile one, its SW teams were accustomed to agile methodology. But incorporating ML teams and the further development of ML in an agile setup presented a new set of challenges due to the experimental and iterative nature of ML development and the challenges associated with communication and collaboration, which makes incorporating agile practices difficult.[9] This resulted in the development of Agile4MLS, a methodology that evolved out of the existing agile practices at FMS to develop ML-enabled systems. The methodology also enabled the organization to develop the product in five months. Since then, the company has been following this practice to develop different ML-enabled systems.

This article highlights key challenges faced by FMS to incorporate agile for development of Proktor, describes the Agile4MLS methodology adopted by us, and further provides key lessons learned and takeaways to the community based on an internal validation study.

The Proktor Application

Proktor (https://www.proktor.com) is an ML-enabled workplace proctoring system that creates a virtual workspace around employees. It uses a webcam of employee laptops to monitor the workplace and further extracts system metrics of employee machines. It makes use of ML on edge to detect any security incidents (e.g., use of smartphones, unauthorized personnel in the workplace, and so on). Privacy is ensured by applying image transformation techniques to store a particular representation of the image, rather than the employee's private workspace image itself. Proktor provides employers/administrators with real-time security profiling of employees and sends notifications in the event of any security breach. It also provides employers with a dashboard view to obtain analytics on any threat incidents and further allows them to customize alerts or threats through a configurable rule engine.

There was a plethora of technical challenges with developing Protkor, such as 1) creating a lightweight desktop client with lightweight ML models, 2) scalability as the clients had at least 10,000 employees, 3) real-time preprocessing of low-quality images, and 4) management of different types of deep learning models. However, apart from technical challenges, it was the development challenges that led to the development of Agile4MLS.

Agile SW Team Versus Nonagile ML Team

Ever since FMS started providing SW services (in 2008), the organization has been predominantly agile, where SCRUM is followed extensively to deliver SW products/services. Since 2018, FMS has had a dedicated ML team. The ML team followed an intensively iterative development style, as in many other organizations.[11] However, as most of the features of Proktor were dependent on ML, close cooperation between the ML and SW teams was necessary. Further, the development time was significantly less. To this end, the following three major challenges motivated us to develop Agile4MLS:

1. The product had to be developed and released in a shorter time interval, and the client required a direct update, thus calling for rapid development.
2. ML development has an experimental nature. This is because there is no notion of one single best algorithm. It depends on availability of data, accuracy, performance, and so forth. Hence, ML development follows an intensively iterative style.[11] Further, the ML team was unaware of agile best practices, and making them work with an SW team won't yield the best possible results due to a lack of knowledge and collaboration problems.[9] The same applies to the SW team, and it was difficult for them to understand the probabilistic and uncertain nature of ML components.[11,12]
3. Most of the SW components had a dependency on ML models. Hence, if the two teams followed different development processes, then SW teams would have to wait for each model application programming interface (API) release from the ML team, which would mean a longer integration time, longer development time, and so on. Further, the API could change due to the experimental nature of ML.

AGILE4MLS Process

The Agile4MLS process is based on the SCRUM practice that was followed at FMS as it was important not to create an entirely new approach for the development of Proktor due to organizational constraints. One of the initial ideas was to create small, cross-functional agile teams with SW and ML team members. However, there were many constraints (as explained in the aforementioned second major challenge) along with the traditional challenges associated with individuals, business, and so forth, as reported in different studies.[13,14]

Moreover, organizations adopting ML have reported the presence of a separate ML team,[4,11] and its development style and pace are different from traditional SW teams. Hence, it was important to keep separation, while at the same time induce agility in the teams. To this end, we made changes at two levels: sprint planning and team organization.

Sprint Planning

Figure 1 shows the sprint planning process of Agile4MLS. The initial set of steps up to extracting features was the same as that of traditional SCRUM, and these were handled by four key stakeholders, namely, the product owner, SW architect, ML architect, and project manager (who also acted as the SCRUM master). However, as opposed to traditional SCRUM, the features were further divided into two main classes: ML-intensive features, which represent those that are mainly dependent on ML techniques, and non-ML-intensive features, which represent those that have little or no dependency on ML. For instance, in Proktor, one of the required features was "employee foreign object detection," which requires ML models to identify objects in the employee workplace, like smartphones. Hence, it was classified as an ML-intensive feature. On the other hand, a feature like "employee information management" was classified as a non-ML-intensive feature.

The ML-intensive features are divided into data stories and model stories by the ML architect. This is because, in ML development, the features may have a direct or indirect relationship to the user instead of traditional user stories.[3] Moreover, from the perspective of the ML team, it's intuitive to think in terms of data that need to be collected, processed, and so on, and the model that needs to be developed to accomplish the feature.[4]

Data Stories. Data stories deal with the collecting, processing, and storing of data required to implement a learning algorithm to accomplish a given ML-intensive feature. We used a few rules to define the data stories. They should consist of the 1) goal, 2) source of data, 3) type of data, and 4) an approximate quantity. Each of the data stories was divided into data tasks. Further, each data task was classified into collection, preprocessing, and management tasks. For example, one of the data stories associated with the ML-intensive feature "employee foreign object detection," was "As a data store, it should provide 1,000 images of phones with varying background and foreground colors." For each data story, the story points are allocated by taking into consideration the most intensive task. In this story, preprocessing each of the 1,000 images requires significant effort (annotation, cleaning, and so forth; each image can take 3–4 min of manual effort). So, this influences story-point estimation.

Model Stories. Model stories deal with the feature extraction, training, testing, and deployment of ML models required to accomplish a given ML-intensive feature. As in the case of data stories, we used a few rules to define the model stories. A model story should consist of the 1) learning goal, 2) type of learning to be used, 3) data source, and 4) expected accuracy. Each model story is categorized into model tasks, which are further divided into four tasks: feature processing, training, validation, and deployment (which includes versioning). For example, one of the model stories in Proktor was "As a learned model, it should detect smartphones in a given image using deep learning with an accuracy of at least 70%." In this case, the story points are estimated by taking into consideration the effort required for training a model for a given story (apart from activities of processing, validation, and so on) and extra points are added to the total by the ML architect to consider any unexpected delays due to the experimental nature of ML.

The non-ML-intensive features, as in traditional SCRUM, was divided into user stories, which were then categorized into two different tasks: 1) a model-integration task, which deals with the integration of an SW component with an ML one, and 2) a traditional task, which is one that does not have any ML dependency. The model-integration task is mapped with the corresponding deployment task of ML stories. All of the ML tasks are moved into the ML sprint, which is then assigned to the ML team. On the other hand, the tasks generated from non-ML-intensive features are moved into the SW sprint and then transferred to the SW team.

These steps helped handle the challenges, that is, the aforementioned first major challenge and some parts of the second major challenge. To address the third major challenge, we introduced the idea of a demo API and 2-sprint ahead in Agile4MLS. The ML team started with tasks that required integration with the SW team two sprints ahead of the SW team. The ML team used this time to do initial research and define the demo API (the actual API that emulates the expected response from an ML component) that the SW team could use for each model-integration task. This further allowed the ML team to take their time in working on building the

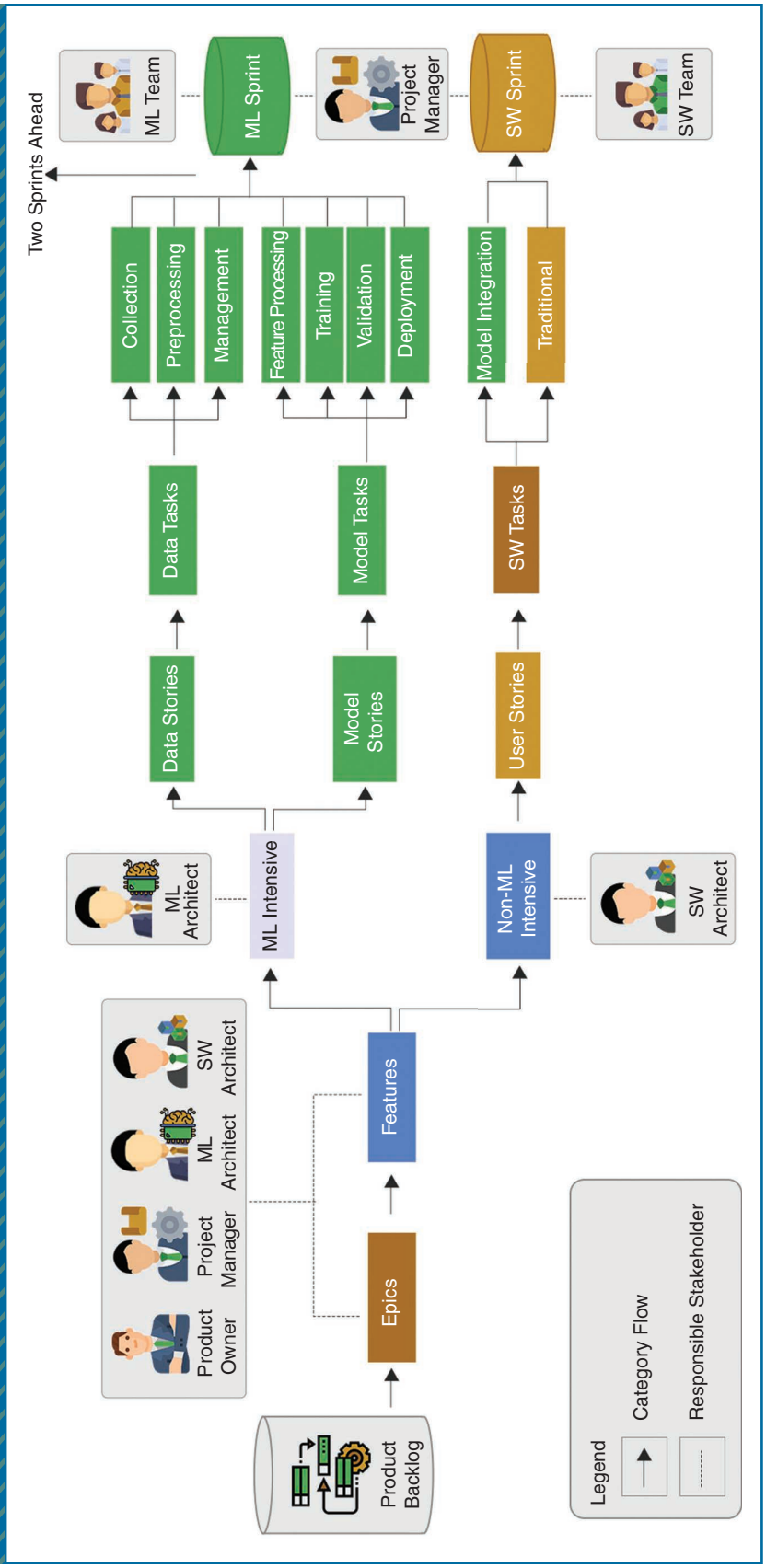

FIGURE 1. The sprint planning process of Agile4MLS.

model that would produce the actual response. To handle the second major challenge, we also introduced changes in the team organization, as explained in the next section.

Team Organization

As explained previously, fundamentally, the entire development team of Proktor was divided into an ML and an SW team based on the members' expertise. The Ops and QA/test personnel were essentially a part of the SW team, although they supported the ML team with all the deployment and validation processes.

As a part of enabling agility, the ML team members had daily stand-up meetings attended by the product owner, ML architect, and all members of the ML team. In addition, one member of the SW team responsible for the model-integration task was also present in these meetings. The member of the SW team attending the ML team meeting changed every sprint depending on who was responsible for handling the model-integration task. The presence of SW team members ensured that the nuances of ML development were understood by the SW team, while on the other hand, the ML team members were able to understand the SW development's best practices gradually.

Stand-up meetings of the SW team are very similar to the traditional SCRUM process, except that the SW team member attending the daily stand-up of the ML team kept the SW team informed about the key issues and progress of the ML team.

To ensure that there is involvement, understanding, and transparency in terms of major decisions among all team members, a weekly sprint stand-up meeting was in place, attended by the four key stakeholders and all members of both the SW

and ML teams. During this meeting, apart from weekly updates made by the members, the product owner gave feedback, and the SW and ML architects presented an overview of major decisions. We noticed that this type of meeting enabled knowledge exchanges, improved collaboration, and guaranteed that every team member was aware of any critical decisions made by either team. This allowed us to easily bridge the gap between the SW team's technical capabilities and the ML team's mathematical capabilities, thereby addressing the aforementioned second major challenge. Moreover, it contributed toward a better synergy between the team members. Both the SW and ML teams were able to appreciate the other's effort and use the other's help in solving problems together as one team.

Further, during the weekly meetings, dedicated time was given for the retrospection of older sprints of both teams, and all team members were encouraged to share their feedback to improve the development processes in the consecutive sprints. This process also helped the architects in making more informed decisions.

Validations and Key Takeaways

Agile4MLS enabled us to release Proktor within the given time of five months (20 iterations). However, to evaluate the effectiveness of the methodology, we conducted an online survey with all 21 members involved in the development of Proktor. Most of the participants had experience working in at least two to four agile projects before Proktor, with four having experience working in more than 10 agile projects. The survey had 32 questions related to participants' work experience, knowledge gained, feedback of Agile4MLS process challenges faced, and suggestions for future improvements. The study design questionnaire, along with responses, is made available online at https://tinyurl.com/4arnr8e7. Table 1 lists the challenges faced by FMS, how they were addressed by Agile4MLS, and the validation of the challenge addressal obtained from practitioners' responses. In general, based on the participants' responses, we have elicited the following lessons learned and takeaways for the community.

- *Agile for ML opens new possibilities*: One of the challenges in the ML development process is

Table 1. Challenges and solutions offered by Agile4MLS, and excerpts from the validation study.

Challenges	Agile4MLS solution	Excerpts from validation
CH1: Shorter time, rapid development	Shorter sprints of one week, ML development through careful sprint planning and splitting of tasks	"I am delighted with the results, and the product came out well. The team worked together well. I will maintain the same agility in the next project also."
CH2: Experimental nature of ML development and collaboration problems	Splitting into model and data stories allows different members to focus on different tasks, thereby allowing faster experimentations. Moreover, 2-sprint ahead allows time for a few experiments before finalizing a demo API. Making the SW team member working on integrations participate in daily meetings of the ML team (this changes) allows for transfer of knowledge and weekly team meetings, including members of the SW and ML teams.	"Agile4MLS enabled the periodic retweaking of the model based on the requirements, keeping data drift in check. The models were continuously tested side by side with the application development that kept improving it over time. Even as the front end of the application was tested, more data was generated, which was used to update the model as well." "The daily meeting helped both teams. The software team became more and more aware of the gray areas of ML results, and they started to anticipate the changes that may come and work, keeping that in mind." "In the weekly meetings, all members of the team will get a brief idea of what's happening in both sides."
CH3: Dependency of most SW components on ML	The demo API and 2-sprint ahead concepts. The ML team uses the two sprints to perform experiments and define an initial demo API (with a dummy response), allowing SW teams to perform a smoother integration with ML components. The ML team can then work on creating the model, which makes the API concrete at a later stage.	"ML development results are not predictable and didn't fit agile development. Earlier, I always waited for the results from the ML team and then started the software development to avoid significant changes that may come in the expected results. In the case of Proktor, we identified and agreed on the ML output changeability factors in the first sprint. Keeping that in mind, we developed the communication protocol, the user experience, etc. The ML development created uncertainty or block for the software team development at no point."

CH1: challenge 1; CH2: challenge 2; CH3: challenge 3.

enabling agility and the difficulty in breaking ML into smaller tasks.[9,11,12] To this end, Agile4MLS provides a mechanism to develop ML-enabled systems by keeping all the principles of agile intact. All the respondents felt that the user stories (data/model stories) were very similar or even better than the traditional SW development process. For instance, one senior respondent mentioned, "It helped us to plan, review, change, and adapt better, which in turn helped us to create quality products based on requirements that were on a very high level." The use of data and model stories, demo APIs, shorter sprints for faster feedback, and the advantage of having weekly and daily meetings were some of the participants' other opinions. However, we need dedicated tools to provide end-to-end support for agile ML. We had to tweak Microsoft Azure Devops (https://azure.microsoft.com/en-us/services/devops/) to implement Agile4MLS (e.g., using tags with colors to annotate model/data stories and corresponding tasks and creating new activities corresponding to ML, such as training, validation, and so forth).

- *Foster better collaboration and communication*: Ensuring smoother collaboration and communication between the SW and ML team members is a big challenge due to various factors.[3,9] Every ML team member agreed that having an SW team member helped them a lot in integration and gave them a better understanding of the overall project and vice versa. For instance, one participant mentioned that "Since I am from an ML background, I was not quite sure how to integrate my code with the back end. Regular meetings from the SW team have helped me toward understanding those." However, another participant from the ML team remarked that it would also be better to have an ML member (handling deployment) participating in daily meetings with the SW team. We believe that the team organization presented in this article can pave the way for gradually having the SW and ML teams work as one cross-functional team.
- *Better handling of uncertainties*: The introduction of ML components brings in a lot of uncertainties due to their probabilistic nature,[4,11] such as issues associated with model drift, data drift, model evolution, and so on.[12,15] These may further affect the SW components consuming the models. Having shorter sprints with regular meetings and interactions helped us avoid the uncertainties to a great extent. One of the respondents felt that using the Agile4MLS process enabled periodic tweaking of the model and verification of the data to keep the data and model drift in check. Another respondent felt that after the time taken for initial model development, with every sprint, the models kept improving, guaranteeing more robustness and contributing to fewer uncertainties. Uncertainties are part and parcel of ML development due to the very nature of ML, and there is a lot of ongoing research in this direction. However, we believe that considering our experience, agile practices can contribute, to a great extent, in managing and reducing them.
- *Toward effective decision making, a better design, and faster release*: One of the key principles of the agile manifesto is to promote better design through regular collaborations. Quite a few respondents felt that understanding how both teams are progressing allowed them to contribute better to different decisions. This also enabled the architects to make informed decisions. The project manager remarked that the development pace was higher than previous projects, which meant that the possible issues of the slowness of ML development were handled effectively in Agile4MLS.

Apart from the takeaways mentioned previously, we also felt that we could better design the system and minimize technical debt. It is also worth noting that we are currently using Agile4MLS at FMS for developing four different types of ML-enabled systems, and the results are very encouraging. We further believe that Agile4MLS can be applied by any organization, in particular, SMEs adopting ML as it provides systematic ways, such as the concept of model/data stories, 2-sprint ahead, demo API, and team organization to develop ML-enabled systems. Moreover, we believe that the takeaways presented in this work can foster academic and practitioner research on the use of agile for better developing ML-enabled systems.

References

1. J. McKendrick. "AI adoption skyrocketed over the last 18 months."

ABOUT THE AUTHORS

KARTHIK VAIDHYANATHAN is an assistant professor with the software engineering research center at the International Institute of Information Technology Hyderabad, Hyderabad, 500032, India. His research interests lie at the intersection of software architecture and machine learning with a specific focus on self-adaptative systems and microservice-based systems. Vaidhyanathan received his Ph.D. in computer science from the Gran Sasso Science Institute. Contact him at karthik.vaidhyanathan@iiit.ac.in.

HENRY MUCCINI is a full professor of software engineering at the University of L'Aquila, L'Aquila, 67100, Italy. His research interests include software architecture descriptions and analysis, adaptive architectures with machine learning, and model-driven engineering applied to software architecture design. Muccini received his Ph.D. in computer science from the University of La Sapienza. Contact him at henry.muccini@univaq.it.

ANISH CHANDRAN is the director of innovation at Founding Minds Pvt. Ltd., Kochi, 682030, India. His research interests include system architecting, imagineering, and artificial intelligence. Chandran received his bachelor's degree in computer science and engineering from Cochin University of Science and Technology. Contact him at anish@foundingminds.com.

REGI ROY is the CEO of Founding Minds Pvt. Ltd., Kochi, 682030, India. His research interests include problem solving around artificial intelligence machine learning/computer vision use cases. Roy received his master's in business administration from Cornell University. Contact him at regi.roy@foundingminds.com.

Harvard Business Review. https://hbr.org/2021/09/ai-adoption-skyrocketed-over-the-last-18-months (Accessed: Mar. 7, 2022).

2. "Survey analysis: Moving AI projects from prototype to production." Gartner. https://www.gartner.com/en/documents/3987202/survey-analysis-moving-ai-projects-from-prototype-to-pro (Accessed: Mar. 7, 2022).
3. H. Muccini, and K. Vaidhyanathan, "Software architecture for ML-based systems: What exists and what lies ahead," in *Proc. IEEE/ACM 1st Workshop AI Eng.-Softw. Eng. AI (WAIN)* May 2021, pp. 121–128, doi: 10.1109/WAIN52551.2021.00026.
4. S. Amershi et al., "Software engineering for machine learning: A case study," in *Proc. 41st Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, 2019, pp. 291–300, doi: 10.1109/ICSE-SEIP.2019.00042.
5. R. Akkiraju et al., "Characterizing machine learning processes: A maturity framework," in *Business Process Management*, D. Fahland, C. Ghidini, J. Becker, M. Dumas, Eds. Cham: Springer-Verlag, 2020, pp. 17–31.
6. V. Lakshmanan, S. Robinson, and M. Munn, *Machine Learning Design Patterns*. Sebastopol, CA, USA: O'Reilly Media, 2020.
7. M. Haakman, L. Cruz, H. Huijgens, and A. van Deursen, "AI lifecycle models need to be revised: An exploratory study in Fintech," *Empirical Softw. Eng.*, vol. 26, no. 5, pp. 1–29, 2021, doi: 10.1007/s10664-021-09993-1.
8. H. B. Braiek and F. Khomh, "On testing machine learning programs," *J. Syst. Softw.*, vol. 164, p. 110,542, Jun. 2020, doi: 10.1016/j.jss.2020.110542.
9. N. Nahar, S. Zhou, G. Lewis, and C. Kästner, "Collaboration challenges in building ML-enabled systems: Communication, documentation, engineering, and process," *Organization*, vol. 1, no. 2, p. 3, 2022, doi: 10.1145/3510003.3510209.
10. J. Bosch, H. H. Olsson, and I. Crnkovic, "Engineering ai systems: A research agenda," in *Artificial*

Intelligence Paradigms for Smart Cyber-Physical Systems, A. K. Luhach and A. Elçi, Eds. Hershey, PA, USA: IGI Global, 2021, pp. 1–19.

11. Z. Wan, X. Xia, D. Lo, and G. C. Murphy, "How does machine learning change software development practices?" *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1857–1871, 2021, doi: 10.1109/TSE.2019.2937083.
12. G. A. Lewis, S. Bellomo, and I. Ozkaya, "Characterizing and detecting mismatch in machine-learning-enabled systems," in *Proc. IEEE/ACM 1st Workshop AI¡¡¡ Eng.-Softw.Eng. AI (WAIN)*, May 2021, pp. 133–140, doi: 10.1109/WAIN52551.2021.00028.
13. Z. Masood, R. Hoda, and K. Blincoe, "Real world scrum a grounded theory of variations in practice," *IEEE Trans. Softw. Eng.*, vol. 48, no. 5, pp. 1579–1591, 2020, doi: 10.1109/TSE.2020.3025317.
14. I. Nurdiani, J. Börstler, S. Fricker, K. Petersen, and P. Chatzipetrou, "Understanding the order of agile practice introduction: Comparing agile maturity models and practitioners' experience," *J. Syst. Softw.*, vol. 156, pp. 1–20, Oct. 2019, doi: 10.1016/j.jss.2019.05.035.
15. A. Cummaudo, S. Barnett, R. Vasa, J. Grundy, and M. Abdelrazek, "Beware the evolving 'intelligent' web service! an integration architecture tactic to guard AI-first components," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 269–280, doi: 10.1145/3368089.3409688.

Digital Object Identifier 10.1109/MS.2022.3210315